



SC3020 Database Systems Principle Group 25 Project 1 Report

Name	Contribution
Ong Hong Xun	20%
Clarence Tan Yan Kai	20%
Wong Rong Jing	20%
Chen ZhongJiang	20%
Chan Zi Jian	20%

College of Computing and Data Science
Nanyang Technological University, Singapore

2025/26 Semester 2

Source: https://github.com/HXong/SC3020_Group25/tree/main/sc3020-project1

Table Of Content

1. Disk Storage.....	3
1.1 Task 1.....	3
1.2 Task 2.....	4
1.3 Task 3.....	6
2. Column 'ctid'.....	7
2.1 Task 4.....	7
2.2 Task 5.....	8
2.3 Task 6.....	13
3. Buffering.....	17
3.1 Task 7.....	17
3.2 Task 8.....	18
3.3 Task 9.....	19
4. Indexing.....	21
4.1 Task 10.....	21
4.2 Task 11.....	23
5. Sophisticated Query Processing.....	26
5.1 Task 12.....	26
5.2 Task 13.....	27
6. Index Maintenance.....	32
6.1 Task 14.....	32
6.2 Task 15.....	33

1. Disk Storage

1.1 Task 1

Using the command:

```
sc3020_project1=# SHOW block_size;
```

Output:

```
block_size
-----
8192
(1 row)
```

This means that each block is 8192 bytes.

1.2 Task 2

Customer:

```
sc3020_project1=# select pg_relation_size('customer');
pg_relation_size
-----
          29491200
(1 row)
```

Size of Relation = 29491200 bytes.

Given each block is 8192 bytes,

Number of Blocks = $29491200 / 8192 = 3600$

LineItem:

```
sc3020_project1=# select pg_relation_size('lineitem');
pg_relation_size
-----
        922419200
(1 row)
```

Size of Relation = 922419200 bytes.

Given each block is 8192 bytes,

Number of Blocks = $922419200 / 8192 = 112600$

Orders:

```
sc3020_project1=# select pg_relation_size('orders');
pg_relation_size
-----
        214106112
(1 row)
```

Size of Relation = 214106112 bytes.

Given each block is 8192 bytes,

Number of Blocks = $214106112 / 8192 = 26136$

Partsupp:

```
sc3020_project1=# select pg_relation_size('partsupp');
pg_relation_size
-----
        143851520
(1 row)
```

Size of Relation = 143851520 bytes.

Given each block is 8192 bytes,

Number of Blocks = $143851520 / 8192 = 17560$

Part:

```
sc3020_project1=# select pg_relation_size('part');
pg_relation_size
-----
          33816576
(1 row)
```

Size of Relation = 33816576 bytes.

Given each block is 8192 bytes,

Number of Blocks = $33816576 / 8192 = 4128$

Supplier:

```
sc3020_project1=# select pg_relation_size('supplier');
pg_relation_size
-----
          1826816
(1 row)
```

Size of Relation = 1826816 bytes.

Given each block is 8192 bytes,

Number of Blocks = $1826816 / 8192 = 223$

Region:

```
sc3020_project1=# select pg_relation_size('region');
pg_relation_size
-----
           8192
(1 row)
```

Size of Relation = 8192 bytes.

Given each block is 8192 bytes,

Number of Blocks = $8192 / 8192 = 1$

Nation:

```
sc3020_project1=# select pg_relation_size('nation');
pg_relation_size
-----
           8192
(1 row)
```

Size of Relation = 8192 bytes.

Given each block is 8192 bytes,

Number of Blocks = $8192 / 8192 = 1$

1.3 Task 3

```
sc3020_project1=# \d+ customer
```

Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description
c_custkey	integer		not null		plain			
c_name	character varying(25)		not null		extended			
c_address	character varying(40)		not null		extended			
c_nationkey	integer		not null		plain			
c_phone	character(15)		not null		extended			
c_acctbal	numeric(15,2)		not null		main			
c_mktsegment	character(10)		not null		extended			
c_comment	character varying(117)		not null		extended			

Indexes:
"customer_pkey" PRIMARY KEY, btree (c_custkey)

Foreign-key constraints:
"fk_customer" FOREIGN KEY (c_nationkey) REFERENCES nation(n_nationkey)

Referenced by:
TABLE "orders" CONSTRAINT "fk_orders" FOREIGN KEY (o_custkey) REFERENCES customer(c_custkey)

Access method: heap

Storage Column:

- plain
 - Used for small, fixed-size data types like integer, boolean and float.
 - Data is always stored directly inline in the table page.
 - No compression, no out-of-line storage.
 - Fastest to access since data is always in the main table.
- main
 - Used for data types like numeric and UUID that prefer to stay inline, but can be moved.
 - PostgreSQL will try to compress the value.
 - Only moves data out-of-line (to TOAST) as an absolute last resort.
- extended
 - The default for large/variable-length types like text, character varying (varchar)
 - PostgreSQL will try to compress the value.
 - Only moves data out-of-line (to TOAST) as an absolute last resort.
 - Most flexible storage strategy

Access method:

- heap
 - Rows are stored in an unordered collection of 8KB pages (blocks)
 - New rows are inserted into any page with free space.
 - There is no inherent physical ordering of rows (unlike a clustered/index-organised table)
 - It is PostgreSQL's default and most common storage method.

2. Column 'ctid'

2.1 Task 4

SELECT ctid, * FROM customer;

Output:

proj1_db=#	SELECT ctid, *	FROM customer;	ctid	c_custkey	c_name	c_address	c_nationkey	c_phone	c_acctbal	c_mktsegment	c_comment
(0,1)	1	Customer#000000001	1	1	Customer#000000001	IVhZlApeRb ot,c,E	15	25-289-701-2988	711.56	BUILDING	to the even, regular platelets. regular, ironic epitaphs nag e
(0,2)	2	Customer#000000002	2	2	Customer#000000002	XSTf4i,NCwDVaIMeStEgynfaRchLXak	13	23-768-687-3665	121.65	AUTOMOBILE	l accounts. blithely ironic theodolites integrate boldly: caref
(0,3)	3	Customer#000000003	3	3	Customer#000000003	ME9k4T0Z0Bhm	1	11-719-790-3364	7498.12	AUTOMOBILE	deposits eat slyly ironic, even instructions. express foxes detect slyly. blithely even accounts abov
(0,4)	4	Customer#000000004	4	4	Customer#000000004	XuS3uLdA6tn	4	14-122-190-6944	2866.83	MACHINERY	requests. final, regular ideas sleep final accou
(0,5)	5	Customer#000000005	5	5	Customer#000000005	KvpyuKpLrB8M6gAIGV85pJz7Tj	3	13-758-562-4364	794.47	HOUSEHOLD	n accounts will have to unwind. foxes cajole accor
(0,6)	6	Customer#000000006	6	6	Customer#000000006	sKZzCsnW7np4d8RrBvv,LR6YUMAH yVn	20	38-114-968-4951	7638.57	AUTOMOBILE	tions. even deposits boost according to the slyly bold packages. final accounts cajole requests. furious
(0,7)	7	Customer#000000007	7	7	Customer#000000007	TdGe5aZNgVpcU5d6vCB9F4asDf6a	18	28-196-982-9789	9561.95	AUTOMOBILE	ainst the ironic, express theodolites. express, even pinto beans among the exp
(0,8)	8	Customer#000000008	8	8	Customer#000000008	IdU1d88d9ymc, 8pRfRCKPjGyG3xcD9wh1S	17	27-147-574-9235	6819.78	BUILDING	among the slyly regular theodolites kindle blithely courts. carefully even theodolites haggle slyly along the ide
(0,9)	9	Customer#000000009	9	9	Customer#000000009	xL1AF7J8uCuFe1eMqFauTrJ5	8	18-138-986-3675	8324.07	FURNITURE	r theodolites according to the requests make thinly excuses: pending requests haggle furiously
(0,10)	10	Customer#000000010	10	10	Customer#000000010	6LrEaV6R6PLVcg12Akt, Q3rqtztc7L v2	5	15-741-346-9870	2783.54	HOUSEHOLD	es regular deposits haggle. fur
(0,11)	11	Customer#000000011	11	11	Customer#000000011	PM5 3K1Kqptzrzvg6318E1	23	33-484-181-3439	272.68	BUILDING	ackages. requests sleep slyly. quickly even pinto beans promise above the slyly regular pinto beans.
(0,12)	12	Customer#000000012	12	12	Customer#000000012	9RMuhtzTz1q	13	23-791-276-1853	3796.49	HOUSEHOLD	to the carefully final brands. blithely regular requests nag. ironic theodolites boast quickly along
(0,13)	13	Customer#000000013	13	13	Customer#000000013	nsYQubVjD79M65hC3SR5p	3	13-761-567-5974	3857.34	BUILDING	ourts sleep carefully after the close frays. carefully bold notornis use ironic requests. blithely
(0,14)	14	Customer#000000014	14	14	Customer#000000014	K0KletML2XQEA	1	11-945-129-3851	5266.39	FURNITURE	, ironic packages across the unus
(0,15)	15	Customer#000000015	15	15	Customer#000000015	Yb8pgxOdLmdo76y, 8ZgUQWLMJ01Y,EC,6dn	23	33-687-542-7681	2788.52	HOUSEHOLD	platelets. regular deposits detect asymptotes. blithely unusual packages nag slyly at the fluff
(0,16)	16	Customer#000000016	16	16	Customer#000000016	cY2eKtZ39AQ2 dWj	18	28-781-669-2487	4881.82	FURNITURE	ly silent courts. thinly regular theodolites sleep fluffily after
(0,17)	17	Customer#000000017	17	17	Customer#000000017	1zrh 8jdtpz3qdU8k0a0C85G5z2rwn#Fdyg87	2	12-978-682-3487	6.34	AUTOMOBILE	packages make! blithely even pint
(0,18)	18	Customer#000000018	18	18	Customer#000000018	3tuG0 AduFu3z7BZ9WvRfnZt	6	16-155-215-1315	5494.43	BUILDING	s sleep. carefully even instructions nag furiously alongside of t
(0,19)	19	Customer#000000019	19	19	Customer#000000019	uc_38H1x84H,wdmL0JvsigUCq2tr	18	28-396-525-5863	8914.71	HOUSEHOLD	nag. furiously careful packages are slyly at the accounts. furiously regular in
(0,20)	20	Customer#000000020	20	20	Customer#000000020	3pKd8npL4Hn	22	32-987-234-8742	7699.48	FURNITURE	g alongside of the special excuses— fluffily entering packages make

SELECT ctid, * FROM lineitem LIMIT 20;

Output:

proj1_db=# SELECT ctid, * FROM lineitem LIMIT 20;																			
ctid	l_orderkey	l_partkey	l_supply	l_linenum	l_quantity	l_extendedprice	l_discount	l_tax	l_returnflag	l_linestatus	l_shipdate	l_commitdate	l_receiptdate	l_shipinstruct	l_shipmode	l_comment			
(0,1)	1	155199	7706	1	17.80	21168.23	0.84	0.82	N	0	1996-03-13	1996-02-12	1996-03-22	DELIVER IN PERSON	TRUCK	regular courts above the			
(0,2)	1	67318	7311	2	36.80	45983.16	0.89	0.86	N	0	1996-04-12	1996-02-28	1996-04-29	TAKE BACK RETURN	MAIL	ly final dependencies: slyly bold			
(0,3)	1	63789	3761	3	8.80	13389.68	0.18	0.82	N	0	1996-01-29	1996-03-05	1996-01-31	TAKE BACK RETURN	REG AIR	riously, regular, express dep			
(0,4)	2	2112	4833	4	28.80	28955.84	0.89	0.86	N	0	1996-04-21	1996-03-38	1996-05-16	NONE	AIR	lites, fluffily even de			
(0,5)	1	26927	1534	5	24.80	23204.48	0.18	0.84	N	0	1996-03-39	1996-03-14	1996-04-01	NONE	FOB	pending foxes, slyly re			
(0,6)	1	15635	638	6	32.80	49638.16	0.87	0.82	N	0	1996-01-38	1996-02-07	1996-02-03	DELIVER IN PERSON	MAIL	carefully slyly ex			
(0,7)	2	186179	1191	1	38.80	44684.46	0.88	0.85	N	0	1997-01-28	1997-01-14	1997-02-02	TAKE BACK RETURN	RAIL	ven requests, deposits breach a			
(0,8)	3	4297	1798	1	45.80	54858.85	0.86	0.88	R	F	1994-02-02	1994-01-04	1994-02-23	NONE	AIR	onside of the furiously brave acco			
(0,9)	3	19836	6540	2	49.80	46796.47	0.18	0.88	R	F	1993-11-09	1993-12-28	1993-11-24	TAKE BACK RETURN	RAIL	unusual accounts, eve			
(0,10)	3	128449	3474	3	27.80	39898.88	0.86	0.87	A	F	1994-01-16	1993-11-22	1994-01-23	DELIVER IN PERSON	SHIP	nal foxes make.			
(0,11)	3	29389	1883	4	2.80	2618.76	0.81	0.86	A	F	1993-12-04	1994-01-07	1994-01-01	NONE	TRUCK	y, fluffily pending d			
(0,12)	3	183895	658	5	28.80	32986.52	0.84	0.88	R	F	1993-12-14	1994-01-18	1994-01-01	TAKE BACK RETURN	FOB	ages nag slyly pending			
(0,13)	3	62145	9662	6	26.80	28733.68	0.18	0.82	A	F	1993-10-29	1993-12-18	1993-11-04	TAKE BACK RETURN	RAIL	ges sleep after the caref			
(0,14)	4	88835	5560	1	38.80	39688.38	0.83	0.88	N	0	1996-01-18	1996-12-14	1996-01-18	DELIVER IN PERSON	REG AIR	- quickly regular packages sleep, idly			
(0,15)	5	188579	8571	1	15.80	23678.55	0.92	0.84	R	F	1994-10-31	1994-08-31	1994-11-28	NONE	AIR	ts make furiously			
(0,16)	5	123927	3928	2	26.80	59723.92	0.87	0.88	R	F	1994-10-16	1994-09-25	1994-10-19	NONE	FOB	sts use slyly quickly special instruc			
(0,17)	5	37531	35	3	58.80	73426.58	0.88	0.83	A	F	1994-08-08	1994-18-13	1994-08-26	DELIVER IN PERSON	AIR	edolites, fluffily unusual			
(0,18)	6	139636	2150	1	37.80	61998.31	0.88	0.83	A	F	1992-04-27	1992-05-15	1992-05-02	TAKE BACK RETURN	TRUCK	p furiously special foxes			
(0,19)	7	182852	9687	1	12.80	13688.68	0.97	0.83	N	0	1996-05-07	1996-03-13	1996-06-03	TAKE BACK RETURN	FOB	ss pinto beans make against th			
(0,20)	7	145243	7785	2	9.80	11594.16	0.88	0.88	N	0	1996-02-01	1996-03-02	1996-02-19	TAKE BACK RETURN	SHIP	es, instructions			

Both tables return a tuple with ctid (0,1) because each table is stored in its own separate data file on disk. The value (0,1) simply indicates the first tuple in the first block of that table's data file.

For the customer table, (0,1) refers to the first customer record (Customer#000000001).

For the lineitem table, (0,1) refers to the first line item record.

These are different physical locations in different files, even though they share the same ctid value format.

2.2 Task 5

customer:

SELECT ctid, c_custkey, c_name FROM customer ORDER BY ctid LIMIT 20;

Output:

```
proj1_db=# SELECT ctid, c_custkey, c_name FROM customer ORDER BY ctid LIMIT 20;
 ctid | c_custkey | c_name
-----+-----+-----
(0,1) |         1 | Customer#000000001
(0,2) |         2 | Customer#000000002
(0,3) |         3 | Customer#000000003
(0,4) |         4 | Customer#000000004
(0,5) |         5 | Customer#000000005
(0,6) |         6 | Customer#000000006
(0,7) |         7 | Customer#000000007
(0,8) |         8 | Customer#000000008
(0,9) |         9 | Customer#000000009
(0,10) |        10 | Customer#000000010
(0,11) |        11 | Customer#000000011
(0,12) |        12 | Customer#000000012
(0,13) |        13 | Customer#000000013
(0,14) |        14 | Customer#000000014
(0,15) |        15 | Customer#000000015
(0,16) |        16 | Customer#000000016
(0,17) |        17 | Customer#000000017
(0,18) |        18 | Customer#000000018
(0,19) |        19 | Customer#000000019
(0,20) |        20 | Customer#000000020
```

The customer table shows tuples stored in primary key order.

ctid (0,1) corresponds to c_custkey = 1 and ctid (0,2) corresponds to c_custkey = 2.

This indicates the table was populated by inserting tuples in ascending c_custkey order, and no major updates/delete operations have reorganized the heap. The physical storage order matches the logical primary key order.

lineitem:

SELECT ctid, l_orderkey, l_partkey, l_suppkey, l_linenum
FROM lineitem ORDER BY ctid LIMIT 20;

Output:

```
proj1_db=# SELECT ctid, l_orderkey, l_partkey, l_suppkey, l_linenumber
proj1_db=# FROM lineitem ORDER BY ctid LIMIT 20;
```

ctid	l_orderkey	l_partkey	l_suppkey	l_linenumber
(0,1)	1	155190	7706	1
(0,2)	1	67310	7311	2
(0,3)	1	63700	3701	3
(0,4)	1	2132	4633	4
(0,5)	1	24027	1534	5
(0,6)	1	15635	638	6
(0,7)	2	106170	1191	1
(0,8)	3	4297	1798	1
(0,9)	3	19036	6540	2
(0,10)	3	128449	3474	3
(0,11)	3	29380	1883	4
(0,12)	3	183095	650	5
(0,13)	3	62143	9662	6
(0,14)	4	88035	5560	1
(0,15)	5	108570	8571	1
(0,16)	5	123927	3928	2
(0,17)	5	37531	35	3
(0,18)	6	139636	2150	1
(0,19)	7	182052	9607	1
(0,20)	7	145243	7758	2

The lineitem table is NOT stored in primary key order.

When ordered by ctid: (0,1) → l_orderkey=1, l_partkey=155190.

When ordered by primary key, the same tuple appears at position 6.

The physical order shows l_orderkey values: 1,1,1,1,1,1,2,3,3,3,3,3,4,5,5,5,6,7,7.

This is typical for large fact tables where data is appended as it arrives, not sorted by the primary key. The ctid order reflects insertion order, which likely follows transaction sequence rather than composite key order. The composite primary key (l_orderkey, l_partkey, l_suppkey, l_linenumber) exists for logical uniqueness, not physical organization.

nation:

```
SELECT ctid, n_nationkey, n_name FROM nation ORDER BY ctid;
```

Output:

```
proj1_db=# SELECT ctid, n_nationkey, n_name FROM nation ORDER BY ctid;
```

ctid	n_nationkey	n_name
(0,1)	0	ALGERIA
(0,2)	1	ARGENTINA
(0,3)	2	BRAZIL
(0,4)	3	CANADA
(0,5)	4	EGYPT
(0,6)	5	ETHIOPIA
(0,7)	6	FRANCE
(0,8)	7	GERMANY
(0,9)	8	INDIA
(0,10)	9	INDONESIA
(0,11)	10	IRAN
(0,12)	11	IRAQ
(0,13)	12	JAPAN
(0,14)	13	JORDAN
(0,15)	14	KENYA
(0,16)	15	MOROCCO
(0,17)	16	MOZAMBIQUE
(0,18)	17	PERU
(0,19)	18	CHINA
(0,20)	19	ROMANIA
(0,21)	20	SAUDI ARABIA
(0,22)	21	VIETNAM
(0,23)	22	RUSSIA
(0,24)	23	UNITED KINGDOM
(0,25)	24	UNITED STATES

The nation table is stored in primary key order.

As a small reference table with only 25 tuples, all rows fit in a single block.

The table was likely populated once in key order and rarely modified, preserving the sequential physical order.

orders:

```
SELECT ctid, o_orderkey, o_custkey FROM orders ORDER BY ctid LIMIT 20;
```

Output:

```
proj1_db=# SELECT ctid, o_orderkey, o_custkey FROM orders ORDER BY ctid LIMIT 20;
```

ctid	o_orderkey	o_custkey
(0,1)	1	36901
(0,2)	2	78002
(0,3)	3	123314
(0,4)	4	136777
(0,5)	5	44485
(0,6)	6	55624
(0,7)	7	39136
(0,8)	32	130057
(0,9)	33	66958
(0,10)	34	61001
(0,11)	35	127588
(0,12)	36	115252
(0,13)	37	86116
(0,14)	38	124828
(0,15)	39	81763
(0,16)	64	32113
(0,17)	65	16252
(0,18)	66	129200
(0,19)	67	56614
(0,20)	68	28547

The orders table is NOT stored in primary key order.

First 7 tuples: o_orderkey = 1,2,3,4,5,6,7 (sequential)

Then jump to: o_orderkey = 32,33,34,35,36,37,38,39

Then jump to: o_orderkey = 64,65,66,67,68

This pattern suggests the data was inserted in batches, not in strict o_orderkey order.

Each batch likely contains a range of order keys, and the physical storage reflects the order of batch insertions rather than the logical key order.

part:

```
SELECT ctid, p_partkey, p_name FROM part ORDER BY ctid LIMIT 20;
```

Output:

```
proj1_db=# SELECT ctid, p_partkey, p_name FROM part ORDER BY ctid LIMIT 20;
```

ctid	p_partkey	p_name
(0,1)	1	goldenrod lavender spring chocolate lace
(0,2)	2	blush thistle blue yellow saddle
(0,3)	3	spring green yellow purple cornsilk
(0,4)	4	cornflower chocolate smoke green pink
(0,5)	5	forest brown coral puff cream
(0,6)	6	bisque cornflower lawn forest magenta
(0,7)	7	moccasin green thistle khaki floral
(0,8)	8	misty lace thistle snow royal
(0,9)	9	thistle dim navajo dark gainsboro
(0,10)	10	linen pink saddle puff powder
(0,11)	11	spring maroon seashell almond orchid
(0,12)	12	cornflower wheat orange maroon ghost
(0,13)	13	ghost olive orange rosy thistle
(0,14)	14	khaki seashell rose cornsilk navajo
(0,15)	15	blanched honeydew sky turquoise medium
(0,16)	16	deep sky turquoise drab peach
(0,17)	17	indian navy coral pink deep
(0,18)	18	turquoise indian lemon lavender misty
(0,19)	19	chocolate navy tan deep brown
(0,20)	20	ivory navy honeydew sandy midnight

The part table is stored in primary key order.

ctid (0,1) → p_partkey=1

ctid (0,2) → p_partkey=2

... ctid (0,20) → p_partkey=20

partsupp:

```
SELECT ctid, ps_partkey, ps_suppkey FROM partsupp ORDER BY ctid LIMIT 20;
```

Output:

```
proj1_db=# SELECT ctid, ps_partkey, ps_suppkey FROM partsupp ORDER BY ctid LIMIT 20;
```

ctid	ps_partkey	ps_suppkey
(0,1)	1	2
(0,2)	1	2502
(0,3)	1	5002
(0,4)	1	7502
(0,5)	2	3
(0,6)	2	2503
(0,7)	2	5003
(0,8)	2	7503
(0,9)	3	4
(0,10)	3	2504
(0,11)	3	5004
(0,12)	3	7504
(0,13)	4	5
(0,14)	4	2505
(0,15)	4	5005
(0,16)	4	7505
(0,17)	5	6
(0,18)	5	2506
(0,19)	5	5006
(0,20)	5	7506

The partsupp table shows partial ordering by primary key.

Grouped by ps_partkey: all entries for partkey 1 appear first (ctid 0,1-0,4)

Then partkey 2 appears (ctid 0,5-0,8)

Then partkey 3 appears (ctid 0,9-0,12)

However, within each partkey group, the suppkey values are not sorted.

The data was inserted grouped by ps_partkey, but not fully sorted by the complete composite primary key.

region:

```
SELECT ctid, r_regionkey, r_name FROM region ORDER BY ctid;
```

Output:

```
proj1_db=# SELECT ctid, r_regionkey, r_name FROM region ORDER BY ctid;
```

ctid	r_regionkey	r_name
(0,1)	0	AFRICA
(0,2)	1	AMERICA
(0,3)	2	ASIA
(0,4)	3	EUROPE
(0,5)	4	MIDDLE EAST

The region table is stored in primary key order.

ctid (0,1) → r_regionkey=0 (AFRICA)

ctid (0,2) → r_regionkey=1 (AMERICA)

ctid (0,3) → r_regionkey=2 (ASIA)

ctid (0,4) → r_regionkey=3 (EUROPE)

ctid (0,5) → r_regionkey=4 (MIDDLE EAST)

supplier:

```
SELECT ctid, s_suppkey, s_name FROM supplier ORDER BY ctid LIMIT 20;
```

Output:

```
proj1_db=# SELECT ctid, s_suppkey, s_name FROM supplier ORDER BY ctid LIMIT 20;
```

ctid	s_suppkey	s_name
(0,1)	1	Supplier#000000001
(0,2)	2	Supplier#000000002
(0,3)	3	Supplier#000000003
(0,4)	4	Supplier#000000004
(0,5)	5	Supplier#000000005
(0,6)	6	Supplier#000000006
(0,7)	7	Supplier#000000007
(0,8)	8	Supplier#000000008
(0,9)	9	Supplier#000000009
(0,10)	10	Supplier#000000010
(0,11)	11	Supplier#000000011
(0,12)	12	Supplier#000000012
(0,13)	13	Supplier#000000013
(0,14)	14	Supplier#000000014
(0,15)	15	Supplier#000000015
(0,16)	16	Supplier#000000016
(0,17)	17	Supplier#000000017
(0,18)	18	Supplier#000000018
(0,19)	19	Supplier#000000019
(0,20)	20	Supplier#000000020

The supplier table is stored in primary key order.

ctid (0,1) → s_suppkey=1 (Supplier#000000001)

ctid (0,2) → s_suppkey=2 (Supplier#000000002)

... ctid (0,20) → s_suppkey=20 (Supplier#000000020)

2.3 Task 6

Before Clustering

```
SELECT ctid, c_custkey, c_phone FROM customer ORDER BY ctid LIMIT 30;
```

Output:

```
proj1_db=# SELECT ctid, c_custkey, c_phone FROM customer ORDER BY ctid LIMIT 30;
```

ctid	c_custkey	c_phone
(0,1)	1	25-989-741-2988
(0,2)	2	23-768-687-3665
(0,3)	3	11-719-748-3364
(0,4)	4	14-128-190-5944
(0,5)	5	13-750-942-6364
(0,6)	6	30-114-968-4951
(0,7)	7	28-190-982-9759
(0,8)	8	27-147-574-9335
(0,9)	9	18-338-906-3675
(0,10)	10	15-741-346-9870
(0,11)	11	33-464-151-3439
(0,12)	12	23-791-276-1263
(0,13)	13	13-761-547-5974
(0,14)	14	11-845-129-3851
(0,15)	15	33-687-542-7601
(0,16)	16	20-781-609-3107
(0,17)	17	12-970-682-3487
(0,18)	18	16-155-215-1315
(0,19)	19	28-396-526-5053
(0,20)	20	32-957-234-8742
(0,21)	21	18-902-614-8344
(0,22)	22	13-806-545-9701
(0,23)	23	13-312-472-8245
(0,24)	24	23-127-851-8031
(0,25)	25	22-603-468-3533
(0,26)	26	32-363-455-4837
(0,27)	27	13-137-193-2709
(0,28)	28	18-774-241-1462
(0,29)	29	10-773-203-7342
(0,30)	30	11-764-165-5076

Before clustering, tuples are stored in c_custkey order.

The phone numbers are scattered randomly across different pages.

For example:

c_custkey=1 has phone 25-989-741-2988

c_custkey=2 has phone 23-768-687-3665

There's no relationship between c_custkey order and phone number order

Create Index and Cluster

```
proj1_db=# CREATE INDEX idx_customer_phone ON customer(c_phone);
CREATE INDEX
proj1_db=# CLUSTER customer USING idx_customer_phone;
CLUSTER
proj1_db=# ANALYZE customer;
ANALYZE
```

After Clustering

SELECT ctid, c_custkey, c_phone FROM customer ORDER BY ctid LIMIT 30;

Output:

```
proj1_db=# SELECT ctid, c_custkey, c_phone FROM customer ORDER BY ctid LIMIT 30;
```

ctid	c_custkey	c_phone
(0,1)	59986	10-100-106-1617
(0,2)	42145	10-100-220-4520
(0,3)	86596	10-100-274-5785
(0,4)	127518	10-100-301-2651
(0,5)	50984	10-100-541-5585
(0,6)	109996	10-100-581-8579
(0,7)	31757	10-100-588-9355
(0,8)	81501	10-100-864-5796
(0,9)	107333	10-101-152-4542
(0,10)	140521	10-101-162-3523
(0,11)	96325	10-101-751-3934
(0,12)	29352	10-101-769-8567
(0,13)	78599	10-101-961-1014
(0,14)	97329	10-101-970-2347
(0,15)	144050	10-102-154-7851
(0,16)	33139	10-102-409-7401
(0,17)	88036	10-102-549-6244
(0,18)	64573	10-102-554-6504
(0,19)	96613	10-102-848-9776
(0,20)	22145	10-103-314-6037
(0,21)	4706	10-103-318-6809
(0,22)	68121	10-103-418-9932
(0,23)	137078	10-103-494-5349
(0,24)	132048	10-103-499-7977
(0,25)	147735	10-103-539-4766
(0,26)	113241	10-103-617-7217
(0,27)	74931	10-103-790-3073
(0,28)	22242	10-103-996-8780
(0,29)	141250	10-104-344-6483
(0,30)	62611	10-104-520-6136

After clustering, the physical order has changed completely:

ctid (0,1) now contains c_custkey=59986 with phone 10-100-106-1617

ctid (0,2) contains c_custkey=42145 with phone 10-100-220-4520

The tuples are now physically ordered by c_phone

Verify Order Matches Phone Order

SELECT ctid, c_custkey, c_phone FROM customer ORDER BY c_phone LIMIT 30;

Output:

```
proj1_db=# SELECT ctid, c_custkey, c_phone FROM customer ORDER BY c_phone LIMIT 30;
```

ctid	c_custkey	c_phone
(0,1)	59986	10-100-106-1617
(0,2)	42145	10-100-220-4520
(0,3)	86596	10-100-274-5785
(0,4)	127518	10-100-301-2651
(0,5)	50984	10-100-541-5585
(0,6)	109996	10-100-581-8579
(0,7)	31757	10-100-588-9355
(0,8)	81501	10-100-864-5796
(0,9)	107333	10-101-152-4542
(0,10)	140521	10-101-162-3523
(0,11)	96325	10-101-751-3934
(0,12)	29352	10-101-769-8567
(0,13)	78599	10-101-961-1014
(0,14)	97329	10-101-970-2347
(0,15)	144050	10-102-154-7851
(0,16)	33139	10-102-409-7401
(0,17)	88036	10-102-549-6244
(0,18)	64573	10-102-554-6504
(0,19)	96613	10-102-848-9776
(0,20)	22145	10-103-314-6037
(0,21)	4706	10-103-318-6809
(0,22)	68121	10-103-418-9932
(0,23)	137078	10-103-494-5349
(0,24)	132048	10-103-499-7977
(0,25)	147735	10-103-539-4766
(0,26)	113241	10-103-617-7217
(0,27)	74931	10-103-790-3073
(0,28)	22242	10-103-996-8780
(0,29)	141250	10-104-344-6483
(0,30)	62611	10-104-520-6136

The ctid order after clustering exactly matches the c_phone order from this query, confirming that CLUSTER successfully reorganized the table.

Why This Happens:

The CLUSTER command physically rewrites the entire table in the order specified by the index. It creates a new copy of the table, sorted by the index key c_phone. After clustering, tuples with the same phone prefix are stored in the same or adjacent blocks, which improves data locality and reduces the number of page reads required for range queries on c_phone.

```
proj1_db=# EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customer WHERE c_phone = '12';
                                QUERY PLAN
-----
Index Scan using idx_customer_phone on customer  (cost=0.42..8.44 rows=1 width=159) (actual time=0.240..0.240 rows=0.00 loops=1)
  Index Cond: (c_phone = '12'::bpchar)
  Index Searches: 1
  Buffers: shared hit=2 read=1
Planning:
  Buffers: shared hit=30 dirtied=4
Planning Time: 0.521 ms
Execution Time: 0.264 ms
```

Performance Implication:

For queries like WHERE c_phone = '12':

- Before clustering: May need to read many scattered pages across the table
- After clustering: All matching tuples are stored consecutively, requiring fewer page reads

3. Buffering

3.1 Task 7

```
test1=# show shared_buffers;
shared_buffers
-----
128MB
(1 row)

test1=#
```

Using the “show shared_buffers;” command, we observed that the buffer size is 128MB . To calculate the number of frames, we take the buffer size(128MB) divided by the block size(8192bytes) obtained in task 1:

$$\text{Number of frames} = \frac{128 \times 1024 \times 1024}{8192} = 16384$$

So there are 16384 frames in the buffer.

3.2 Task 8

```
test1=# explain analyze select * from lineitem;
                                         QUERY PLAN
-----
Seq Scan on lineitem (cost=0.00..172621.32 rows=6002132 width=117) (actual time=1.395..1109.533 rows=6001215.00 loops=1)
  Buffers: shared read=112600
Planning:
  Buffers: shared hit=100 read=5
Planning Time: 7.904 ms
Execution Time: 1382.720 ms
(6 rows)

test1=# explain analyze select * from lineitem;
                                         QUERY PLAN
-----
Seq Scan on lineitem (cost=0.00..172621.32 rows=6002132 width=117) (actual time=0.193..552.743 rows=6001215.00 loops=1)
  Buffers: shared hit=94 read=112506
Planning Time: 0.133 ms
Execution Time: 834.945 ms
(4 rows)

test1=#
```

In this task, we observed that in the first query execution,

- Hit = 0
- Miss = 112600

As this is the first query after the reboot, the buffer is expected to be empty. PostgreSQL would need to read the blocks from the disk into the buffer thus resulting in a high buffer miss rate.

In the second query execution, we observe that

- Hit = 94
- Miss = 112506

As this is the second query execution, some of these pages were already cached into the shared buffer after the first query execution thus resulting in more hits. As the buffer is significantly smaller than the lineitem table, it can only store a portion of the pages while the remaining will be read from the disk thus explaining the high buffer miss.

3.3 Task 9

```
test1=# explain analyze select * from lineitem where l_orderkey=70;
               QUERY PLAN
-----
Index Scan using lineitem_pkey on lineitem (cost=0.43..33.19 rows=15 width=117) (actual time=0.044..0.048 rows=6.0
 loops=1)
  Index Cond: (l_orderkey = 70)
  Index Searches: 1
  Buffers: shared hit=4
  Planning Time: 0.161 ms
  Execution Time: 0.080 ms
(6 rows)

test1=# explain analyze select * from lineitem where l_orderkey=70;
               QUERY PLAN
-----
Index Scan using lineitem_pkey on lineitem (cost=0.43..33.19 rows=15 width=117) (actual time=0.046..0.049 rows=6.0
 loops=1)
  Index Cond: (l_orderkey = 70)
  Index Searches: 1
  Buffers: shared hit=4
  Planning Time: 0.160 ms
  Execution Time: 0.080 ms
(6 rows)

test1=# explain analyze select * from lineitem where l_orderkey=71;
               QUERY PLAN
-----
Index Scan using lineitem_pkey on lineitem (cost=0.43..33.19 rows=15 width=117) (actual time=0.046..0.049 rows=6.0
 loops=1)
  Index Cond: (l_orderkey = 71)
  Index Searches: 1
  Buffers: shared hit=4
  Planning Time: 0.167 ms
  Execution Time: 0.079 ms
```

In the first query, the number of hit and miss are

- Hit = 4
- Miss = 0

In the second query, the number of hit and miss are

- Hit = 4
- Miss = 0

In the third query, the number of hit and miss are

- Hit = 4
- Miss = 0

In the first query, the relevant index and heap pages were already stored in the memory so there is no need for any disk read. Since the second query is identical, it simply needs to access the same cached pages again so the buffer hit is the same as the first query.

For the third query, even if the value is different, it still uses the same index pages that were stored in the buffer. This shows that PostgreSQL reuses cached pages efficiently and the b-tree index reduces the number of pages needed for primary key lookup.

4. Indexing

4.1 Task 10

A)

We will first use `d+` command to find the name of the B-tree index on the PK of *customer*.

```
\d+ customer
```

Output:

Table "public.customer"								
Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description
c_custkey	integer		not null		plain			
c_name	character varying(25)		not null		extended			
c_address	character varying(40)		not null		extended			
c_nationkey	integer		not null		plain			
c_phone	character(15)		not null		extended			
c_acctbal	numeric(15,2)		not null		main			
c_mktsegment	character(10)		not null		extended			
c_comment	character varying(117)		not null		extended			
Indexes:								
"customer_pkey" PRIMARY KEY, btree (c_custkey) CLUSTER								
Foreign-key constraints:								
"fk_customer" FOREIGN KEY (c_nationkey) REFERENCES nation(n_nationkey)								
Referenced by:								
TABLE "orders" CONSTRAINT "fk_orders" FOREIGN KEY (o_custkey) REFERENCES customer(c_custkey)								
Access method: heap								

From the output, we can see that "customer_pkey" is the PK, so we can use it for the clustering.

```
cluster customer using customer_pkey;
analyze customer;
```

Added analyze as planner might still think the data is randomly distributed after cluster.

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customer WHERE c_custkey = 100;
```

Explain buffers shows the buffer behaviour (buffer hits/misses)

```
QUERY PLAN
-----
Index Scan using customer_pkey on customer (cost=0.42..8.44 rows=1 width=159) (actual time=0.058..0.059 rows=1 loops=1)
  Index Cond: (c_custkey = 100)
  Buffers: shared hit=1 read=3
Planning:
  Buffers: shared hit=36 read=1
Planning Time: 0.234 ms
Execution Time: 0.101 ms
(7 rows)
```

The query results in an Index Scan using the PK index customer_pkey.

The execution plan shows: "Buffers: shared hit=1 read=3"

Since c_custkey is the PK, the predicate is highly selective (exactly one tuple). Hence, PostgreSQL uses a B-tree index to directly locate the matching row.

The buffer stats indicate that 3 pages were read from disk and 1 page was already in memory. These pages correspond to B-tree traversal (root, internal, leaf) and the heap page containing the tuple.

This demonstrates that equality predicates on PK are efficiently handled using B-tree indexes.

B)

```
CREATE INDEX idx_customer_custkey_hash ON customer USING hash (c_custkey);
ANALYZE customer;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customer WHERE c_custkey = 100;
```

Execution Output:

```
QUERY PLAN
-----
Index Scan using idx_customer_custkey_hash on customer (cost=0.00..8.02 rows=1 width=159) (actual time=0.083..0.084 rows=1 loops=1)
  Index Cond: (c_custkey = 100)
  Buffers: shared hit=3
Planning:
  Buffers: shared hit=35 dirtied=7
Planning Time: 0.571 ms
Execution Time: 0.135 ms
(7 rows)
```

After creating a hash index on c_custkey, the query plan changes to:

“Index Scan using idx_customer_custkey_hash”

“Index Cond: (c_custkey = 100)”

Since the predicate is an equality condition, PostgreSQL selected the hash index. Hash indexes are optimized for equality searches and do not maintain sorted order. The estimated cost (0.00..8.02) as compared to B-tree index (0.42..8.44) is slightly lower, which explains why the planner prefers using the hash index.

The buffer stats show “shared hit=3”, indicating that all required pages were already in memory, and no disk I/O was required.

This demonstrates that hash indexes are more suitable for equality predicates compared to B-tree indexes.

C)

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customer WHERE c_custkey >= 200
AND c_custkey <= 201;
```

Execution Output:

```
----- QUERY PLAN -----
Index Scan using customer_pkey on customer (cost=0.42..8.46 rows=2 width=159) (actual time=0.009..0.011 rows=2 loops=1)
  Index Cond: ((c_custkey >= 200) AND (c_custkey <= 201))
  Buffers: shared hit=4
Planning:
  Buffers: shared hit=18
Planning Time: 0.803 ms
Execution Time: 0.050 ms
(7 rows)
```

For the range query, PostgreSQL uses B-tree indexes instead of hash indexes. Evidently:

“Index Scan using customer_pkey”

“Index Cond: ((c_custkey >= 200) AND (c_custkey <= 201))”

Even though a hash index exists on c_custkey, it is not used. This is because hash indexes only support equality predicates and do not maintain sorted order. Range queries require ordered traversal of index keys, which is supported by b-tree indexes.

Therefore, the planner chooses the B-tree PK index to evaluate the range condition instead of hash indexes.

4.2 Task 11

A)

```
cluster customer using customer_pkey;
create index idx_customer_phone on customer(c_phone);
ANALYZE customer;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customer;
```

Execution output:

```
----- QUERY PLAN -----
Seq Scan on customer (cost=0.00..5097.00 rows=150000 width=159) (actual time=0.008..25.145 rows=150000 loops=1)
  Buffers: shared hit=3597
Planning:
  Buffers: shared hit=32 read=2
Planning Time: 1.771 ms
Execution Time: 33.128 ms
(6 rows)
```

PostgreSQL uses Sequential Scan even though clustered + created the phone index.

The query requests all rows (“rows=150000”). Using an index would mean scanning the index (many pages) then fetching almost every heap page. Additionally, extra overhead cost also incurred due to random access. Hence, a sequential scan is cheaper for full table reads.

B)

```
explain (ANALYZE, BUFFERS) select * from customer where c_phone >= '2';
```

Execution Output:

```
sc3020_project1=# explain (analyze, buffers) select * from customer where c_phone >= '2';
                                QUERY PLAN
-----
Seq Scan on customer (cost=0.00..5472.00 rows=90487 width=159) (actual time=0.105..84.908 rows=89923 loops=1)
  Filter: (c_phone >= '2'::bpchar)
  Rows Removed by Filter: 60077
  Buffers: shared hit=3597
Planning:
  Buffers: shared hit=3
Planning Time: 1.573 ms
Execution Time: 91.624 ms
(8 rows)
```

For this query, PostgreSQL also uses a Sequential Scan, even though there is an index on c_phone.

Execution plan shows:

- "rows = 89923"
- "Rows Removed by Filter: 60077"
- "Total table size: (89923+60077 = 150000)"

Since approximately 60% of the table satisfies the condition, the predicate has low selectivity. Using the index would require retrieving a large number of tuple pointers and performing many random heap accesses, which is more expensive compared to scanning the entire table sequentially.

C)

```
SELECT * FROM customer WHERE c_phone >= '2' AND c_phone <= '21';
```

Execution Output:

```
                                QUERY PLAN
-----
Bitmap Heap Scan on customer (cost=157.91..3973.84 rows=5999 width=159) (actual time=9.149..22.366 rows=6009 loops=1)
  Recheck Cond: ((c_phone >= '2'::bpchar) AND (c_phone <= '21'::bpchar))
  Heap Blocks: exact=2950
  Buffers: shared hit=2950 read=26
-> Bitmap Index Scan on idx_customer_phone (cost=0.00..156.41 rows=5999 width=0) (actual time=8.541..8.542 rows=6009 loops=1)
   Index Cond: ((c_phone >= '2'::bpchar) AND (c_phone <= '21'::bpchar))
   Buffers: shared read=26
Planning:
  Buffers: shared hit=3
Planning Time: 0.545 ms
Execution Time: 23.045 ms
(11 rows)
```

PostgreSQL uses a Bitmap Index Scan followed by a Bitmap Heap Scan.

The Bitmap Index Scan first traverses the B-tree index `idx_customer_phone` to identify matching tuple pointers and constructs a bitmap of heap page locations. The Bitmap Heap Scan then retrieves the required heap pages in sorted order, reducing random disk I/O.

Since the predicate matches around 6009 rows (“rows=6009”), 4% of the table, it is selective enough to justify using the index but not sufficiently enough for a simple index scan. The bitmap strategy balances index usage and heap access efficiency.

The buffer statistics also show that only a small number of index pages were read from disk (Buffers: shared read=26), while most heap pages were already in memory (Buffers: shared hit=2950 read=26). This demonstrates how bitmap scanning reduces random I/O compared to a standard index scan.

This demonstrates that PostgreSQL chooses between Sequential Scan, Index Scan, and Bitmap Scan based on predicate selectivity.

D)

```
cluster customer using idx_customer_phone;
ANALYZE customer;
SELECT * FROM customer WHERE c_phone >= '2' AND c_phone <= '21';
```

Execution Output:

```
QUERY PLAN
-----
Bitmap Heap Scan on customer  (cost=157.91..3972.83 rows=5999 width=159) (actual time=2.108..3.832 rows=6009 loops=1)
  Recheck Cond: ((c_phone >= '2'::bpchar) AND (c_phone <= '21'::bpchar))
  Heap Blocks: exact=146
  Buffers: shared hit=146 read=26
    -> Bitmap Index Scan on idx_customer_phone  (cost=0.00..156.41 rows=5999 width=0) (actual time=2.018..2.042 rows=6009 loops=1)
        Index Cond: ((c_phone >= '2'::bpchar) AND (c_phone <= '21'::bpchar))
        Buffers: shared read=26
Planning:
  Buffers: shared hit=22 read=2
Planning Time: 1.889 ms
Execution Time: 4.550 ms
(11 rows)
```

After clustering the customer on `idx_customer_phone`, the number of heap blocks accessed during the Bitmap Heap Scan decreased dramatically from 2950 to 146.

Buffer statistics show:

- “Heap Blocks: exact=146”
- “Buffers: shared read=26” (heap pages)
- “Buffers: shared read=26” (index pages)

Compared to pre-clustering execution, where 2950 heap pages were accessed, clustering significantly improves data locality. Since the table is now physically ordered by `c_phone`, tuples satisfying the range predicate are stored close together, reducing the number of heap pages required.

As a result, execution time improved from 23ms to 4.5ms, showing the effectiveness of clustering for range queries on indexed attributes.

5. Sophisticated Query Processing

5.1 Task 12

A)

```
create index idx_customer_phone on customer(c_phone);
cluster customer using idx_customer_phone;
ANALYZE customer;
explain (ANALYZE, BUFFERS) select * from customer where c_phone >= '2' and
c_phone <= '3' and
c_custkey >= 10000 and c_custkey <= 11000;
```

Execution Output:

```
----- QUERY PLAN -----
Bitmap Heap Scan on customer  (cost=22.23..2212.73 rows=389 width=159) (actual time=0.811..1.502 rows=407 loops=1)
  Recheck Cond: ((c_custkey >= 10000) AND (c_custkey <= 11000))
  Filter: ((c_phone >= '2'::bpchar) AND (c_phone <= '3'::bpchar))
  Rows Removed by Filter: 594
  Heap Blocks: exact=856
  Buffers: shared hit=856 read=6
    -> Bitmap Index Scan on customer_pkey  (cost=0.00..22.13 rows=971 width=0) (actual time=0.177..0.200 rows=1001 loops=1)
      Index Cond: ((c_custkey >= 10000) AND (c_custkey <= 11000))
      Buffers: shared read=6
Planning:
  Buffers: shared hit=31 read=2
Planning Time: 1.384 ms
Execution Time: 1.794 ms
(13 rows)
```

For this query, PostgreSQL chooses a Bitmap Index Scan on the PK index of customer_pkey.

For PK range (“c_custkey >= 10000 AND c_custkey <= 11000”): it is 11000-10000+1=1001 out of 150000, making it very selective (0.67%)

In comparison to Phone condition (“c_phone >= '2' AND c_phone <= '3'”): it is likely more than 1000 rows since we achieved 6000 rows for ‘2’ <= c_phone <= ‘21’

So, the planner chooses the PK index first.

During execution, it found 1001 rows via PK range, then heap scan checked for phone condition. It also removed 594 rows from the phone filter, which 407 remained.

This demonstrates PostgreSQL’s cost-based optimisation strategy. It selects the index with the lowest estimated cost rather than combining multiple indexes unnecessarily.

B)

```
explain (ANALYZE, BUFFERS) select * from customer where c_phone >= '2' and
c_phone <= '20-170' and c_custkey >= 10000 and c_custkey <= 11000;
```

Execution Output:

```
sc3020_project1=# explain (analyze, buffers) select * from customer where c_phone >= '2' and c_phone <= '20-170' and
c_custkey >= 10000 and c_custkey <= 11000;

                                QUERY PLAN
-----
Index Scan using idx_customer_phone on customer  (cost=0.42..35.67 rows=3 width=159) (actual time=0.308..0.461 rows=6 loops=1)
  Index Cond: ((c_phone >= '2'::bpchar) AND (c_phone <= '20-170'::bpchar))
  Filter: ((c_custkey >= 10000) AND (c_custkey <= 11000))
  Rows Removed by Filter: 443
  Buffers: shared hit=12 read=4
Planning Time: 0.446 ms
Execution Time: 0.539 ms
(7 rows)
```

In this follow-up query, PostgreSQL switches to using the index `idx_customer_phone` instead of the PK index. This is because the predicate (`"c_phone >= '2' AND c_phone <= '20-170'"`) is highly selective.

From buffer statistics

- "Rows = 3"

This implies that the phone range is much more selective than `"10000 <= c_custkey <= 11000"`, as given earlier to be 1000 rows.

Since `c_phone` is a string attribute, comparisons are lexicographical, and the upper bound `'20-170'` restricts the range significantly.

The execution plan shows:

"Index Scan using `idx_customer_phone`"

The execution plan shows only 6 rows returned. As the phone condition is now more selective than the PK range, PostgreSQL uses the phone index and applies the `c_custkey` condition as a filter.

5.2 Task 13

```
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'lineitem';
```

Execution Output:

indexname	indexdef
lineitem_pkey	CREATE UNIQUE INDEX lineitem_pkey ON public.lineitem USING btree (l_orderkey, l_partkey, l_suppkey, l_linenumbe

(1 row)

A)

```
explain (ANALYZE, BUFFERS) select * from lineitem where l_orderkey = 2000;
```

Execution Output:

```
QUERY PLAN
-----
Index Scan using lineitem_pkey on lineitem (cost=0.43..33.20 rows=15 width=117) (actual time=1.241..1.256 rows=0 loops=1)
  Index Cond: (l_orderkey = 2000)
  Buffers: shared read=3
Planning:
  Buffers: shared hit=58 read=1
Planning Time: 3.737 ms
Execution Time: 1.279 ms
(7 rows)
```

The query uses an Index Scan on the primary-key B-tree (lineitem_pkey). A B-tree lookup requires traversing from root to the internal node and to the leaf page, then fetching the matching tuple from the heap. The execution shows a very small number of buffer page accesses ("Buffers: shared read=3"), indicating that the index traversal touches only a few index pages, consistent with a shallow B-tree.

B)

```
explain (ANALYZE, BUFFERS) select * from lineitem where l_orderkey <= 2000
and l_partkey >= 10000 and l_partkey <= 10010;
```

Execution Output:

```
QUERY PLAN
-----
Index Scan using lineitem_pkey on lineitem (cost=0.43..68.07 rows=1 width=117) (actual time=1.389..1.866 rows=1 loops=1)
  Index Cond: ((l_orderkey <= 2000) AND (l_partkey >= 10000) AND (l_partkey <= 10010))
  Buffers: shared hit=4 read=7
Planning:
  Buffers: shared hit=2 read=2
Planning Time: 6.992 ms
Execution Time: 2.381 ms
(7 rows)
```

Step 1: B-tree traversal

PostgreSQL performs an Index Scan on lineitem_pkey

1. Starts at root of B-tree
2. Traverses internal nodes
3. Reaches the leaf level

As the first indexed column is l_orderkey, and the predicate includes (l_orderkey <= 2000), Postgres can use the composite index

Step 2: Range scan on first column

Since l_orderkey has a range condition and not equality

- The scan must cover all index entries where l_orderkey is from up to 2000.
- It cannot jump directly to a single key
- It scans a range of index entries

Step 3: Applying second-column condition

The index definition order is: {l_orderkey, l_partkey, ...}

As the first column is a range not equality, Postgres cannot efficiently use l_partkey as a narrowing prefix key.

Instead:

- It scans all matching entries for l_orderkey <= 2000
- Then applies the l_partkey condition while scanning
- Effectively filtering within the index scan

So all conditions appear under Index Cond.

Step 4: Heap fetch

For each matching index entry:

- Postgres fetches the corresponding heap tuple
- Verifies conditions
- Returns matching rows

In our buffer query, 1 row matched the execution.

C)

```
explain (ANALYZE, BUFFERS) select * from lineitem where l_partkey >= 10000
and l_partkey <= 10010;
```

Execution Output:

```

                                QUERY PLAN
-----
Gather  (cost=1000.00..151143.09 rows=285 width=117) (actual time=46.339..2058.981 rows=300 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=1 read=112599
-> Parallel Seq Scan on lineitem  (cost=0.00..150114.59 rows=119 width=117) (actual time=30.084..2021.897 rows=100 loops=3)
   Filter: ((l_partkey >= 10000) AND (l_partkey <= 10010))
   Rows Removed by Filter: 2000305
   Buffers: shared hit=1 read=112599
Planning Time: 1.002 ms
JIT:
  Functions: 6
  Options: Inlining false, Optimization false, Expressions true, Deforming true
Timing: Generation 2.502 ms, Inlining 0.000 ms, Optimization 3.411 ms, Emission 63.230 ms, Total 69.143 ms
Execution Time: 2400.463 ms
(14 rows)
```

Step 1: Index not used

Postgres did not use lineitem_pkey due to composite index rule:

- A composite B-tree index can only be used efficiently if the first column(s) are constrained.
- Index order is (l_orderkey, l_partkey, ...)
- But query constraints only l_partkey BETWEEN ...

The first column (l_orderkey) is not constrained. Therefore, composite B-trees cannot be used.

Step 2: Sequential Scan Chosen

Since no usable index exists, Postgres chooses "Parallel Seq Scan on lineitem"

This implies:

- Entire table scanned
- Every row checked
- Rows not satisfying condition are discarded

Step 3: Parallel Execution

Because lineitem is large, Postgres parallelizes:

- "Workers Planned: 2"
- "Workers Launched: 2"

So, there are 3 processes total, 1 leader and 2 workers. Each scans part of the table and the results are gathered at the top node.

Step 4: Massive I/O

"Buffers: shared read=112599"

"Rows removed by Filter: 2000305"

"Execution Time: 2400ms"

This shows:

- Entire table scanned
- Huge number of pages read
- Many rows filtered out
- Much slower than index scan

D)

```
explain (ANALYZE, BUFFERS) select l_partkey from lineitem where l_partkey
>= 10000 and l_partkey <= 10010;
```

Execution Output:

```

                                QUERY PLAN
-----
Index Only Scan using lineitem_pkey on lineitem (cost=0.43..152418.62 rows=285 width=4) (actual time=5.408..685.399 rows=300 loops=1)
  Index Cond: ((l_partkey >= 10000) AND (l_partkey <= 10010))
  Heap Fetches: 0
  Buffers: shared hit=11 read=23009
Planning Time: 0.284 ms
JIT:
  Functions: 2
  Options: Inlining false, Optimization false, Expressions true, Deforming true
  Timing: Generation 0.653 ms, Inlining 0.000 ms, Optimization 0.582 ms, Emission 4.620 ms, Total 5.855 ms
Execution Time: 686.415 ms
(10 rows)
```

The query uses an Index Only Scan on lineitem_pkey because the projection includes l_partkey only, which is stored in the PK index itself. Hence, Postgres can return results directly from the index and performs 0 heap fetches. However, since l_partkey is not the leading column of the composite index (l_orderkey, l_partkey, ...), the index is not ordered by l_partkey, so the predicate cannot be satisfied by a tight index range scan. As a result, Postgres still needs a large number of index pages (shared read = 23009) and filters entries during the scan. This shows that index-only scans can reduce heap I/O when all required columns are in the index, but column order still affects how efficiently predicates can be applied.

E)

```
explain (ANALYZE, BUFFERS) select * from lineitem where l_partkey >= 10000
and l_partkey <= 10000;
```

Execution Output:

```
----- QUERY PLAN -----
Index Scan using lineitem_pkey on lineitem (cost=0.43..152470.34 rows=32 width=117) (actual time=18.417..670.844 rows=14 loops=1)
  Index Cond: ((l_partkey >= 10000) AND (l_partkey <= 10000))
  Buffers: shared hit=10 read=23020 written=1
Planning Time: 0.311 ms
JIT:
  Functions: 2
  Options: Inlining false, Optimization false, Expressions true, Deforming true
  Timing: Generation 0.880 ms, Inlining 0.000 ms, Optimization 0.000 ms, Emission 0.000 ms, Total 0.880 ms
Execution Time: 671.959 ms
(9 rows)
```

Although the query uses an equality condition on `l_partkey`, Postgres still performs an Index Scan on the composite PK index (`l_orderkey`, `l_partkey`, `l_suppkey`, `l_linenum`). Since `l_partkey` is not the leading column of the index, entries with the same `l_partkey` value are not stored contiguously in the B-tree. As a result, Postgres must scan a large portion of the index and filter entries, leading to a high number of index page reads (23020 buffers).

As the query selects all columns, Postgres must also fetch heap tuples, so it cannot use an Index Only Scan. This shows that even equality conditions cannot be efficiently supported if they are not the first column in the composite index.

6. Index Maintenance

Setup for both tasks:

PostgreSQL in running in docker container

Python 3.10.11 with psycopg==3.3.3, psycopg-binary==3.3.3

6.1 Task 14

Code:

```
import time
import random

import psycopg # PostgreSQL database adapter for Python

# 1. Open a connection
conn = psycopg.connect(
    host="localhost",
    port=5432,
    dbname="sc3020_project1",
    user="postgres",
    password="postgres"
)

#2. Issue the bulk query
with conn.cursor() as cur:
    cur.execute("DROP TABLE IF EXISTS t1;")
    cur.execute("CREATE TABLE t1(a integer, b integer, c integer, d integer, e
integer, primary key(a, b, c, d, e));")
    cur.execute("CREATE INDEX idx_t1_a on t1(a);")
    cur.execute("CREATE INDEX idx_t1_b on t1(b);")
    cur.execute("CREATE INDEX idx_t1_c on t1(c);")
    cur.execute("CREATE INDEX idx_t1_d on t1(d);")
    cur.execute("CREATE INDEX idx_t1_e on t1(e);")
    start = time.perf_counter()
    cur.execute("INSERT INTO t1 SELECT (floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int,(floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int,(floor(random()*10000)+1)::int FROM
GENERATE_SERIES(1, 1000000);")
    end = time.perf_counter()
```



```

print(f"Bulk Time: {end - start} seconds")
cur.execute("COMMIT;")

#3. Issue many individual queries
with conn.cursor() as cur:
    start = time.perf_counter()
    for i in range(1000000):
        a = random.randint(1, 10000)
        b = random.randint(1, 10000)
        c = random.randint(1, 10000)
        d = random.randint(1, 10000)
        e = random.randint(1, 10000)

        # issue a statement here to insert a tuple (a, b, c, d, e)
        cur.execute("INSERT INTO t1 VALUES (%s, %s, %s, %s, %s);", (a, b, c, d,
e))

    end = time.perf_counter()
    print(f"Individual Time: {end - start} seconds")

# 4. Close the connection
conn.close()

```

Source code:

https://github.com/HXong/SC3020_Group25/blob/main/sc3020-project1/python/task14.py

Execution:

```

(.venv) PS C:\Users\Anjay\Desktop\SC3020_Group25> python -u "c:\Users\Anjay\Desktop\SC3020_Group25\sc3020-project1\task14.py"
Bulk Time: 15.604912500362843 seconds
Individual Time: 543.6027966002002 seconds

```

Sending individual requests takes significantly longer due to overheads involved in parsing and planning each insertion.

6.2 Task 15

Code:

```

import time
import random

import psycopg # PostgreSQL database adapter for Python

```

```

# 1. Open a connection
conn = psycopg.connect(
    host="localhost",
    port=5432,
    dbname="sc3020_project1",
    user="postgres",
    password="postgres"
)

#2. Issue the bulk query on a table without indexes
with conn.cursor() as cur:
    cur.execute("DROP TABLE IF EXISTS t2;")
    cur.execute("CREATE TABLE t2(a integer, b integer, c integer, d integer, e
integer, primary key(a, b, c, d, e));")
    start = time.perf_counter()
    cur.execute("INSERT INTO t2 SELECT (floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int,(floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int,(floor(random()*10000)+1)::int FROM
GENERATE_SERIES(1, 100000);")
    end = time.perf_counter()
    print(f"Index-less Bulk Time: {end - start} seconds")
    cur.execute("COMMIT;")

# 3. Close the connection
conn.close()

```

Source code:

https://github.com/HXong/SC3020_Group25/blob/main/sc3020-project1/python/task15.py

Execution:

```

(.venv) PS C:\Users\Arjay\Desktop\SC3020_Group25> python -u "c:\Users\Arjay\Desktop\SC3020_Group25\sc3020-project1\task15.py"
Index-less Bulk Time: 4.118017599917948 seconds

```

Inserting on a table without indexes is significantly faster due to not needing to update the indexes incrementally, according to [PostgreSQL documentation](#).